

Aufgabenstellung: Systemdokumentation des eigenen Projekts (Docs-as-Code)

Ausgangslage

Ihr eigenes Projekt hat in der Implementierung einen Meilenstein erreicht. Die Architektur Ihres Systems folgt einem **Modularen Monolithen** (analog zum Schulbibliotheks-Beispiel) und besteht aus folgenden Hauptkomponenten:

- **Mobile-Client** (für Endnutzer/Kunden)
- **Admin-Client** (für Systemadministratoren/Mitarbeiter)
- **Spring-Backend** (zentrale Geschäftslogik & Ressourcen-API)
- **Keycloak-Server** (für Identitäts- und Zugriffsmanagement)
- **Datenbank** (für die Persistenz der Fachdaten)

Hinsichtlich Zugriffsverwaltung und Identity Management (IAM) sind für die beiden Clients bereits folgende Prozesse funktionsfähig implementiert:

Mobile-Client:

- Registrierung (Self-Service) inklusive E-Mail-Verifizierung
- Login (SSO über Keycloak)
- Passwort vergessen (Passwort-Reset-Link per E-Mail)
- Logout

Admin-Client:

- Vorab-Registrierung durch Keycloak-Administratoren (User erhält temporäres Passwort)
- Erster Login (Zwang zum Setzen eines neuen, eigenen Passworts)
- Passwort vergessen (Passwort-Reset-Link per E-Mail)
- Logout

Ihre Aufgabe ist es nun, den aktuellen Stand Ihres Projekts nach dem **Docs-as-Code**-Ansatz zu dokumentieren. Die Dokumentation soll direkt in Ihrem Projekt-Repository abgelegt werden.

Aufgaben

Aufgabe 1: Systemübersicht & Kontextdiagramm (C4 Level 1)

Beschreiben Sie Ihr System aus der Vogelperspektive.

- **Visualisierung:** Erstellen Sie ein Kontextdiagramm (**Mermaid** oder **PlantUML**), das die externen Akteure (Ihre spezifischen Endnutzer und Admins) und das System (als Blackbox) zeigt. Der Mailing-Provider (für E-Mail-Verifizierung/Passwort-Resets) sollte als externes Fremdsystem ebenfalls abgebildet sein.
- **Beschreibung:** Formulieren Sie 1-2 Sätze zum primären Einsatzzweck Ihres Systems.

Aufgabe 2: Bausteinsicht & Architektur (C4 Level 2)

Geben Sie einen Einblick in die Architektur und die Verteilung der Verantwortlichkeiten.

- **Visualisierung:** Erstellen Sie ein Container-Diagramm, das den Mobile-Client, den Admin-Client, das Spring-Backend, den Keycloak-Server und die Datenbank sowie deren Kommunikationspfade (z.B. REST, OAuth2/OIDC, SMTP) zueinander zeigt.
- **Beschreibung:** Erstellen Sie für das **Spring-Backend** und den **Keycloak-Server** eine Kurzbeschreibung (Verantwortlichkeiten, öffentliche Schnittstellen, Abhängigkeiten gemäß Skript-Template).

Aufgabe 3: Laufzeitsicht / Sequenzdiagramme für IAM-Prozesse

Dokumentieren Sie die dynamischen Prozesse Ihres Identity-Managements.

Wählen Sie **zwei** der folgenden Use-Cases aus und erstellen Sie dafür jeweils ein detailliertes Sequenzdiagramm (**Mermaid**):

1. **Self-Service Registrierung Mobile:** Ablauf vom Absenden des Formulars über Keycloak bis zur E-Mail-Verifizierung.
2. **Admin-Onboarding:** Ablauf vom ersten Login mit dem temporären Passwort bis zum erzwungenen Setzen des neuen Passworts in Keycloak.
3. **Passwort-Reset-Flow:** Einleitung des "Passwort vergessen"-Ablaufs über einen der Clients bis hin zur erfolgreichen Neuvergabe des Passworts.
4. **Login & API-Call:** (Standard-SSO-Flow) Token-Austausch über Keycloak und ein anschließender Request an einen geschützten Endpunkt Ihres Spring-Backends inkl. Token-Validierung.

Aufgabe 4: Domain- und Datenmodell

Dokumentieren Sie den Kern der Fachdomäne Ihres individuellen Projekts (die Daten, die in Ihrem Spring-Backend verarbeitet werden).

- **Visualisierung:** Erstellen Sie ein Domain-Klassendiagramm (**PlantUML** oder **Mermaid**), das Ihre zentralen Entitäten abbildet (z.B. Kunden, Bestellungen, Artikel – je nach Ihrem Projektthema).
- **Beschreibung:** Fügen Sie eine textuelle Beschreibung der wichtigsten Entitäten und Aggregate-Grenzen an.

Aufgabe 5: Entwickler-Setup & lokales Deployment

Schreiben Sie einen kurzen Guide für die Inbetriebnahme Ihres Projekts für neue Entwickler.

- Zeigen Sie die notwendigen Schritte oder **docker-compose** Befehle, um Ihre gesamte Umgebung (Clients, Spring-Boot, Keycloak inkl. importiertem Realm, DB) lokal zu starten (- basierend auf den von Ihnen gewählten Konzepten aus dem Bereich Deployment).
- Beschreiben Sie kurz, unter welchen lokalen URLs die Clients und der Keycloak zu erreichen sind und wie geprüft werden kann, ob das Backend läuft (z.B. **/actuator/health** oder eine Basis-API).

Anforderungen an die Abgabe (Deliverables)

1. **Ordnerstruktur:** Legen Sie im Root-Verzeichnis Ihres Repositories einen Ordner **/docs/architecture/** an.
2. **Einstiegspunkt:** Erstellen Sie dort eine zentrale **README.md** (ähnlich der arc42-Struktur), die als Inhaltsverzeichnis dient und inhaltlich auf Unterdokumente (z.B. **01_context.md**,

`02_building_blocks.md`, etc. oder als Gesamt-Dokument mit Ankern) strukturiert ist.

3. **Docs-as-Code Paradigma:** Die Dokumentation darf **keine importierten Bilddateien** (z.B. `.png`, `.jpg`, `.drawio`) für die Diagramme enthalten. Alle Visualisierungen müssen direkt als Codeblock (`mermaid` oder `plantuml`) im Markdown geschrieben sein.
4. **Qualität:** Achten Sie auf eine saubere technische Sprache. Die Diagramme sollen nachvollziehbar sein und zur textuell untermauert werden. Vermeiden Sie Lücken (z.B. wenn in Task 3 der Ablauf beschrieben wird, aber die Akteure nicht aus dem Kontext C1 Task 2/Systemgrenzen aus Task 1 & 2 fehlen).